

each of the slides in order. The first two characters of the header will be the image number.

```
00
Start the python application my_photos from the terminal

01
A new image will be displayed to you.

02
Type the text you would like to appear as a caption in the text box.

03
Once you press enter, the text will be displayed on the image as you can see.

04
Now click on the save and next button. The image will be saved.

05
And you will be shown the next picture.
Repeat the steps until all the photographs are processed.
Note, that if you do not wish to put a caption on a picture and save it, you can press the next button.
```

The implementation

The core logic of your application will be as follows:

```
#!/usr/bin/env python
import os, sys
import wave
import Image, ImageTk, ImageDraw

script_file = open('Script.txt')
# iterate over each scene
for scene_id, image, text in scene_data(script_file):
    duration = text_to_speech(text)
    # create frames assuming 25 frames per sec
    for frame_no in range(25*duration):
        image.save(scene_id + "%03d"%frame_no + ".jpg")
    # convert the frames into a scene
    os.system('mencoder -audiofile ' + scene_id + '.text.wav -oac mp3lame
"mf,/' +
        + scene_id + ".jpg" -mf fps=25 -o out_' +
        + scene_id + '.avi -ovc lavc -lavcopts vcodec=mpeg4')
    # Create an animated scene to end using the last image
    animated_scene(image)
# combine the scenes into a single film
os.system('mencoder -ovc copy -oac mp3lame -o output.avi out_*.avi')
```

The script file is opened. It's best for the scene ID to be a numeric string of a fixed number of digits. That will ensure the order of scenes is easily maintained.

An image and the corresponding text are selected. The text is converted to a speech file. The image is copied as many times as the number of frames that will be

needed for the duration of the speech file.

The speech file and the images (using the `mf://xx*`, `jpg` URL) are combined and converted into an AVI file by using Mencoder. The sound file is converted to an MP3. If you are familiar with FFmpeg, you may use that instead of Mencoder.

Finally, all the AVI files are combined into a single AVI file.

The code for the generator to fetch the image and the text file will be as follows:

```
def scene_data(script_file):
    while True:
        # the first two characters in the script are the scene id
        scene_id = script_file.readline()[0:2]
        # readline will return an empty string after EOF
        if scene_id.strip() == "":
            break
        # the images are png files in screencast subdirectory
        im_file = 'screencast/PhotoApp' + scene_id + '.png'
        image = Image.open(im_file)
        frame = image.resize((640,480))
        # read lines until an empty line
        text = ""
        while True:
            line = script_file.readline()
            if line.strip() == "":
                break
        # Append replacing new line by a space
        text += line.replace("\n", " ")
        yield scene_id, frame, text
```

The script file structure was explained above. The code keeps reading the script file until there is no more data. The first two characters of the header line are the scene ID. The images must be named as per a fixed format with two characters being the scene ID.

The image is resized to a fixed size. The generator yields the values of the scene ID, the resized image and the text associated with that image.

The next step is the code to convert the text to speech.

```
def text_to_speech(text):
    # uncomment ESPEAK or FESTIVAL command and system call
    # ESPEAK = 'espeak -w text.wav -s120 %s'
    # os.system(ESPEAK % (text))
    FESTIVAL = 'echo %s | text2wave -o text.wav -F 44100 -scale 2.0'
    os.system(FESTIVAL % (text))
    win = wave.open('text.wav')
    # modify the wave file to add a short silence
    # before the start and at the end
    wout = wave.open(scene_id + '.text.wav', 'w')
    # create the wave file with same parameters in the input file
    wout.setchannels(win.getchannels())
    wout.setsampwidth(win.getsampwidth())
    wout.setframerate(win.getframerate())
    # half a second of silence
```